

Redis Ansible Scripts - Version: 3.0

This file documents the scripts and plays provided by this package. In most cases, the name of the shell script corresponds to the name of the toplevel playbook which it executes. The toplevel playbook will invoke one or more plays defined in the 'roles' directory structure. In some cases, a top-level playbook may execute multiple roles.

The use of the 'roles' structure allows plays to be mixed and matched in toplevel playbooks.

The plays are implemented using a recommended [Ansible Inventory Structure](#). In this scheme, Redis Enterprise clusters are equivalent to Ansible groups. Cluster level variables are found in a group_var file, while node level variables will be found in host_var files. These files are hosted in subdirectories 'group_vars' and 'host_vars' beneath the 'inventory/' subdir.

```
|  ├── all_clusters.yml
|  ├── group_vars
|  |   ├── all
|  |   ├── rocky8_clstr.yml
|  |   ├── rocky8_spot.yml
|  |   ├── rocky_nodes.yml
|  |   ├── ubuntu_nodes.yml
|  |   └── ubuntu_spot.yml
|  ├── host_vars
|  |   ├── tf-mc-rocky8-1.yml
|  |   ├── tf-mc-rocky8-2.yml
|  |   ├── tf-mc-rocky8-3.yml
|  |   ├── tf-r8spot.yml
|  |   └── tf-ubspot.yml
|  ├── rocky8_clstr.yml
|  ├── rocky8_spot.yml
|  ├── rocky_clusters.yml
|  ├── single_nodes.yml
|  └── ubuntu_spot.yml
```

The top-level .yml files name the correct host and group vars. This allows a top level inventory file to maintain multiple RE clusters. This directory structure can be ported to Ansible Tower or AAP2.

The example configuration files show examples of single node clusters, multinode clusters as well as examples for managing multiple clusters from single top-level inventory. Refer to official [Ansible documentation](#) for detailed information on creating the required inventory files. For cloud, and cloud-like, implementations, where host creation and deletion can be very dynamic, refer to working with [dynamic inventories](#).

In order to make these examples more 'portable', the base location for the package, as well as the location of the inventory files can be set using the script 'setbase.sh' in the root directory of the package. In this way, if there are existing Ansible inventory definitions, the plays can be directed to them by setting the 're_inv' environmental variable.

The scripts for running the example plays are located in 'scripts/', the top-level playbooks are located in 'playbooks/'. The roles containing the individual tasks are in 'playbooks/roles/'.

NOTE:

These scripts are provided as working examples of how to use Ansible to provision and maintain Redis Enterprise clusters and databases.

The scripts were developed by the Redis Professional Services team, but they are not officially supported. The scripts will almost certainly require modification for use in production environments. For example, most enterprises will require enhancements for security - the cluster credentials in these examples appear in clear text in the 'group_vars' files.

In practice the credentials would be sourced from a secure source, such as Hashicorp Vault.

Similarly, in practice, there would likely be more error checking and retry logic. These examples are kept as simple as possible for clarity - they are not intended to be production grade since each enterprise will have their own, often very specific, requirements.

Customers have used these examples as a basis for implementation in Ansible Tower. That implementation is beyond the scope of this package.

Scripts used to execute provided plays

script	use	notes
redis-add-ldap-mapping.sh	map ldap group to role defined in Redis	roles are identified by role id - (redis-list-ldap-mapping.sh to display ids)
redis-create-cluster.sh	Creates the cluster using the API	Driven by env set by source.sh
redis-create-crdb.sh	Creates CRDBs	CRDB defined by json body in ./CRDBS
redis-create-database.sh	create a single redis database	example input - databases/walnut_single.json
redis-create-databases.sh	create a multiple redis databases	example input - databases/databases.yaml
redis-create-local-user.sh	create local user in Redis control plane	example input - localusers/mikec.yaml
redis-delete-crdb.sh	Delete a CRDB	requires CRDB id;use crdb-cli or redis-list-crdb.sh to obtain id

script	use	notes
redis-delete-database	Delete database using bdb_id	bdb_id can be found using 'rladmin status'
redis-delete-db-name.sh	Delete database by name	case-sensitive - must match exactly
redis-delete-ldap-mapping.sh	delete ldap mapping using mapping id	redis-list-ldap-mapping.sh to display ids
redis-delete-local-user.sh	delete local control plane user	example input - localusers/mikec.yaml (only 'name' is required)
redis-install.sh	install redislabs enterprise on a set of nodes	Used with Redis Enterprise version 5.x through 7.2
redis-ldap-update.sh	update ldap configuration	example input - json/ldap.json
redis-list-crdb.sh	List all CRDBs on a cluster	requires only cluster FQDN and login credentials
redis-list-ldap-mapping.sh	list both ldap mappings and redis roles	provides IDs for roles and mappings
redis-print-vars	print ansible vars for specified host	useful for debugging
redis-print-version.sh	Prints version tags from group_vars/all/main.yaml	for version control
redis-quorum_node.sh	Set quorum node by node id	must be done before provisioning databases
redis-setup-singlenode-clstr.sh	Install RE, create cluster, update license	example of multiple roles in single playbook
redis-uninstall.sh	removes redislabs from the nodes	uses rl_uninstall.sh
redis-update-certs.sh	updates the certificates for the cluster	verified, cleaned up. format for directory changed.
redis-update-cluster.sh	update cluster config via API call and json body	example shows imbedded json in the role
redis-update-database.sh	Update db configuration using input yaml	requires bdb_id
redis-update-license.sh	updates the license file	license selected by env var

script	use	notes
redis-update-local-user.sh	Update local control plane user using input yaml	requires user uid
redis-updt-ldap-mapping.sh	Update mapping of ldap group to RE role	requires map_uid
redis-upgrade.sh	upgrades the cluster, NOT databases	apply new version of Redis Enterprise
redis-upgrade-databases.sh	upgrades both the CRDBs and the BDBs to the latest version	to be run after a cluster upgrade